

République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



École Nationale Supérieure d'Informatique Sidi-Bel-Abbès

Fiche TP N° 02 :

Feature extraction and embeddings

Dadoune Ikram

Groupe 1 - IASD -

Année Universitaire 2024-2025

Contents

1	A. Préparation de données	3
1.1	Importation des données	3
1.2	Affichage des valeurs manquantes	3
1.3	Analyse de la distribution des classes	3
2	B. Encodage de la variable à prédire (facultatif)	4
3	C. Construction des bases d'entraînement et de test	5
3.1	Division du dataset	5
3.2	Analyse du déséquilibre des classes	5
4	D. Méthodes de vectorisation	6
4.1	Lexical Frequency (CountVectorizer)	6
4.2	One-Hot Encoding (Binary Representation)	6
4.3	TF-IDF (Term Frequency - Inverse Document Frequency)	6
5	E. Entraînement	7
5.1	Création des modèles MLPClassifier	7
5.2	Entraînement des modèles	7
5.3	Prédiction et évaluation des performances	7
5.4	Résultats attendus	8
6	F. TEST	9
6.1	1. Prédire les classes en appliquant les trois modèles sur les trois représentations de test . . .	9
6.2	2. Afficher le rapport de classification en utilisant les mesures de performance (accuracy, precision, recall...)	9
6.3	3. Calculer le temps de prédiction pour chaque modèle	10
6.4	Conclusion	10
7	G. Vectorisations basées sur les embeddings de mots	11
7.1	1. Word2Vec	11
7.2	2. GloVe	11
7.3	3. FastText	12
8	H. Entraînement / Test	13
8.1	1. Les mêmes questions des sections E et F mais avec les nouvelles représentations vectorielles présentées dans la section G	13
9	Comparaison des modèles	14
10	Analyse des méthodes de vectorisation	14
11	Conclusion	14
12	Justification	14

1 A. Préparation de données

1.1 Importation des données

Nous importons le fichier de données prétraité "cleaneddata.csv" en utilisant la bibliothèque pandas :

```
1 import pandas as pd
2
3 path = "cleaned_data.csv"
4 df = pd.read_csv(path)
5 df.head(10)
```

1.2 Affichage des valeurs manquantes

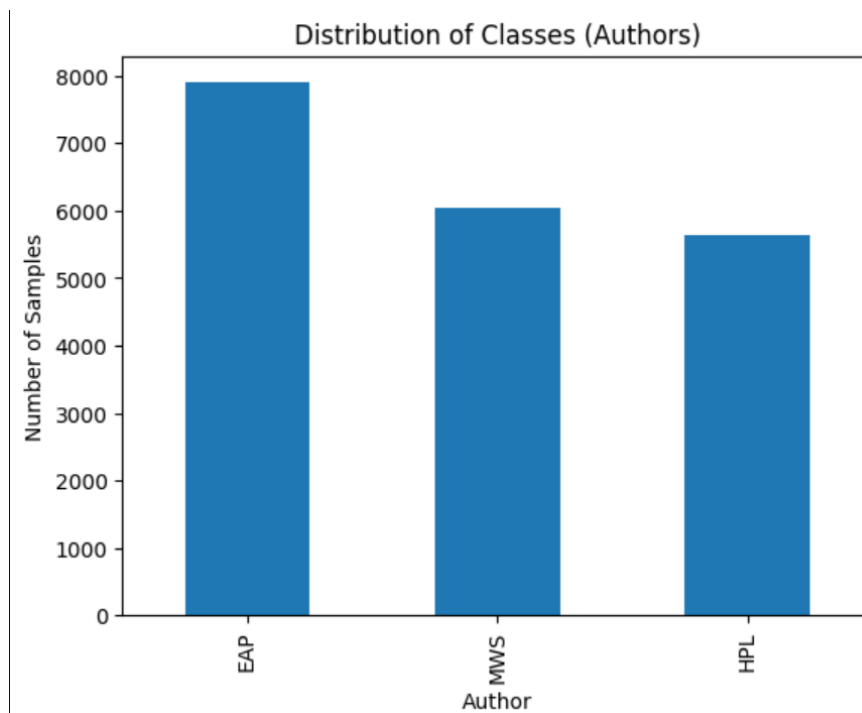
Nous vérifions s'il existe des valeurs manquantes dans le dataset et nous les supprimons si nécessaire :

```
1 print(df.isnull().sum())
2 df = df.dropna()
```

1.3 Analyse de la distribution des classes

Nous vérifions la répartition des classes pour identifier un éventuel déséquilibre :

```
1 import matplotlib.pyplot as plt
2
3 author_counts = df['author'].value_counts()
4 print(author_counts)
5
6 author_counts.plot(kind='bar')
7 plt.xlabel("Auteur")
8 plt.ylabel("Nombre d' échantillons ")
9 plt.title("Distribution des classes")
10 plt.show()
```



2 B. Encodage de la variable à prédire (facultatif)

Nous utilisons l'encodage OneHot pour transformer les labels :

```
1 from sklearn.preprocessing import OneHotEncoder
2 import pandas as pd
3
4 encoder = OneHotEncoder(sparse_output=False)
5
6 encoded_labels = encoder.fit_transform(df[['author']])
7
8 encoded_df = pd.DataFrame(encoded_labels, columns=encoder.categories_[0]).astype(int)
9
10 df = pd.concat([df, encoded_df], axis=1).drop(columns=['author'])
11
12 df.head(10)
```

	id	cleaned_text	EAP	HPL	MWS
0	id26305	process however aforded aforded means ascerta...	1.0	0.0	0.0
1	id17569	never ocured fumbling might mere mistake	0.0	1.0	0.0
2	id11008	left hand gold snuff box capered hill cutting ...	1.0	0.0	0.0
3	id27763	lovely spring loked windsor terace sixteen fer...	0.0	0.0	1.0
4	id12958	finding nothing else even gold superintendent ...	0.0	1.0	0.0
5	id22965	youth pased solitude best years spent gentle f...	0.0	0.0	1.0
6	id09674	astronomer perhaps point took refuge suggestio...	1.0	0.0	0.0
7	id13515	surcingle hung ribands body	1.0	0.0	0.0
8	id19322	knew could say stereotomy without brought thin...	1.0	0.0	0.0
9	id00912	confess neither structure lanquages code gover...	0.0	0.0	1.0

3 C. Construction des bases d'entraînement et de test

3.1 Division du dataset

Nous divisons le dataset en deux parties : entraînement et test, en utilisant `train_test_split` avec une taille de test de 30% et `random_state=0` :

```
1 from sklearn.model_selection import train_test_split
2
3 X = df[['cleaned_text']]
4 y = df[['EAP', 'HPL', 'MWS']]
5
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
7
8 train_df = pd.concat([X_train, y_train], axis=1)
9 test_df = pd.concat([X_test, y_test], axis=1)
10
11 print("Training set size:", train_df.shape)
12 print("Test set size:", test_df.shape)
```

Résultat attendu :

Training set size: (13701, 4)

Test set size: (5872, 4)

3.2 Analyse du déséquilibre des classes

Le dataset étant déséquilibré, nous stratifions les échantillons afin d'obtenir une répartition similaire dans chaque classe :

```
1 from imblearn.over_sampling import RandomOverSampler
2 import pandas as pd
3
4 y = df["author"]
5 X = df[["cleaned_text"]]
6
7 oversampler = RandomOverSampler(random_state=0)
8
9 X_resampled, y_resampled = oversampler.fit_resample(X, y)
10
11 df_resampled = pd.DataFrame({"cleaned_text": X_resampled["cleaned_text"], "author": y_resampled})
12
13 print(df_resampled["author"].value_counts())
```

Résultat attendu :

```
author
EAP    7896
HPL    7896
MWS    7896
Name: count, dtype: int64
```

4 D. Méthodes de vectorisation

4.1 Lexical Frequency (CountVectorizer)

Nous utilisons la méthode de fréquence lexicale pour vectoriser le dataset d'entraînement et de test :

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 bow_vectorizer = CountVectorizer()
4
5 X_train_bow = bow_vectorizer.fit_transform(X_train['cleaned_text'])
6 X_test_bow = bow_vectorizer.transform(X_test['cleaned_text'])
7
8 print("BoW Training Data Shape:", X_train_bow.shape)
9 print("BoW Test Data Shape:", X_test_bow.shape)
```

Résultat attendu :

BoW Training Data Shape: (13701, 22209)

BoW Test Data Shape: (5872, 22209)

4.2 One-Hot Encoding (Binary Representation)

Nous appliquons l'encodage one-hot sur les données textuelles :

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 onehot_vectorizer = CountVectorizer(binary=True)
4
5 X_train_onehot = onehot_vectorizer.fit_transform(X_train['cleaned_text'])
6 X_test_onehot = onehot_vectorizer.transform(X_test['cleaned_text'])
7
8 print("One-Hot Encoding Training Shape:", X_train_onehot.shape)
9 print("One-Hot Encoding Test Shape:", X_test_onehot.shape)
```

Résultat attendu :

One-Hot Encoding Training Shape: (13701, 22209)

One-Hot Encoding Test Shape: (5872, 22209)

4.3 TF-IDF (Term Frequency - Inverse Document Frequency)

Nous entraînons un modèle de vectorisation TF-IDF sur les données d'entraînement :

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 tfidf_vectorizer = TfidfVectorizer()
4
5 X_train_tfidf = tfidf_vectorizer.fit_transform(X_train['cleaned_text'])
6 X_test_tfidf = tfidf_vectorizer.transform(X_test['cleaned_text'])
7
8 print("TF-IDF Matrix Shape (Training Data):", X_train_tfidf.shape)
9 print("TF-IDF Matrix Shape (Test Data):", X_test_tfidf.shape)
10
11 print("First 10 words in TF-IDF Vocabulary:", tfidf_vectorizer.get_feature_names_out()[:10])
```

Résultat attendu :

TF-IDF Matrix Shape (Training Data): (13701, 22209)

TF-IDF Matrix Shape (Test Data): (5872, 22209)

First 10 words in TF-IDF Vocabulary: ['ab' 'aback' 'abandon' 'abandoned' 'abandoning' 'abandonment' 'ab' 'abasement' 'abashment' 'abate']

5 E. Entrainement

5.1 Création des modèles MLPClassifier

Nous créons trois modèles MLPClassifier avec différentes représentations vectorielles :

```
1 from sklearn.neural_network import MLPClassifier
2
3 mlp_bow = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=0)
4 mlp_onehot = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=0)
5 mlp_tfidf = MLPClassifier(hidden_layer_sizes=(100,), alpha=0.01, max_iter=300, random_state=0)
```

5.2 Entrainement des modèles

Nous entraînons ces modèles sur les trois représentations vectorielles :

```
1 mlp_bow.fit(X_train_bow, y_train)
2 mlp_onehot.fit(X_train_onehot, y_train)
3 mlp_tfidf.fit(X_train_tfidf, y_train)
```

5.3 Prédiction et évaluation des performances

Nous évaluons la performance des modèles sur les données de test :

```
1 from sklearn.metrics import accuracy_score, classification_report
2
3 # BoW Representation
4 y_test_pred_bow = mlp_bow.predict(X_test_bow)
5 accuracy_bow = accuracy_score(y_test, y_test_pred_bow)
6 print(f"BoW Representation Test Accuracy: {accuracy_bow:.4f}")
7 print(classification_report(y_test, y_test_pred_bow))
8
9 # One-Hot Encoding
10 y_test_pred_onehot = mlp_onehot.predict(X_test_onehot)
11 accuracy_onehot = accuracy_score(y_test, y_test_pred_onehot)
12 print(f"\nOne-Hot Encoding Test Accuracy: {accuracy_onehot:.4f}")
13 print(classification_report(y_test, y_test_pred_onehot))
14
15 # TF-IDF
16 y_test_pred_tfidf = mlp_tfidf.predict(X_test_tfidf)
17 accuracy_tfidf = accuracy_score(y_test, y_test_pred_tfidf)
18 print(f"\nTF-IDF Test Accuracy: {accuracy_tfidf:.4f}")
19 print(classification_report(y_test, y_test_pred_tfidf))
```

5.4 Résultats attendus

BoW Representation Training Performance:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	5528
1	1.00	1.00	1.00	3944
2	1.00	1.00	1.00	4229
micro avg	1.00	1.00	1.00	13701
macro avg	1.00	1.00	1.00	13701
weighted avg	1.00	1.00	1.00	13701
samples avg	1.00	1.00	1.00	13701

One-Hot Encoding Training Performance:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	5528
1	1.00	1.00	1.00	3944
2	1.00	1.00	1.00	4229
micro avg	1.00	1.00	1.00	13701
macro avg	1.00	1.00	1.00	13701
weighted avg	1.00	1.00	1.00	13701
samples avg	1.00	1.00	1.00	13701

TF-IDF Training Performance:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	5528
1	1.00	1.00	1.00	3944
2	1.00	1.00	1.00	4229
micro avg	1.00	1.00	1.00	13701
macro avg	1.00	1.00	1.00	13701
weighted avg	1.00	1.00	1.00	13701
samples avg	1.00	1.00	1.00	13701

6 F. TEST

6.1 1. Prédire les classes en appliquant les trois modèles sur les trois représentations de test

```
1 from sklearn.metrics import accuracy_score, classification_report
2
3 # BoW Representation
4 y_test_pred_bow = mlp_bow.predict(X_test_bow)
5 accuracy_bow = accuracy_score(y_test, y_test_pred_bow)
6 print(f"BoW Representation Test Accuracy: {accuracy_bow:.4f}")
7 print(classification_report(y_test, y_test_pred_bow))
8
9 # One-Hot Encoding
10 y_test_pred_onehot = mlp_onehot.predict(X_test_onehot)
11 accuracy_onehot = accuracy_score(y_test, y_test_pred_onehot)
12 print(f"\n One-Hot Encoding Test Accuracy: {accuracy_onehot:.4f}")
13 print(classification_report(y_test, y_test_pred_onehot))
14
15 # TF-IDF
16 y_test_pred_tfidf = mlp_tfidf.predict(X_test_tfidf)
17 accuracy_tfidf = accuracy_score(y_test, y_test_pred_tfidf)
18 print(f"\n TF-IDF Test Accuracy: {accuracy_tfidf:.4f}")
19 print(classification_report(y_test, y_test_pred_tfidf))
```

6.2 2. Afficher le rapport de classification en utilisant les mesures de performance (accuracy, precision, recall...)

BoW Representation Test Accuracy: 0.7580

	precision	recall	f1-score	support
EAP	0.75	0.77	0.76	2369
HPL	0.79	0.73	0.76	1690
MWS	0.74	0.76	0.75	1813
accuracy			0.76	5872
macro avg	0.76	0.76	0.76	5872
weighted avg	0.76	0.76	0.76	5872

One-Hot Encoding Test Accuracy: 0.7563

	precision	recall	f1-score	support
EAP	0.75	0.77	0.76	2369
HPL	0.79	0.73	0.76	1690
MWS	0.74	0.76	0.75	1813
accuracy			0.76	5872
macro avg	0.76	0.75	0.76	5872
weighted avg	0.76	0.76	0.76	5872

TF-IDF Test Accuracy: 0.7807

	precision	recall	f1-score	support
EAP	0.78	0.80	0.79	2369
HPL	0.80	0.75	0.78	1690

MWS	0.76	0.78	0.77	1813
accuracy			0.78	5872
macro avg	0.78	0.78	0.78	5872
weighted avg	0.78	0.78	0.78	5872

6.3 3. Calculer le temps de prédiction pour chaque modèle

```

1 import time
2
3 # Measure prediction time for BoW
4 start_time = time.time()
5 y_test_pred_bow = mlp_bow.predict(X_test_bow)
6 bow_time = time.time() - start_time
7 print(f" Prediction Time (BoW): {bow_time:.4f} seconds")
8
9 # Measure prediction time for One-Hot Encoding
10 start_time = time.time()
11 y_test_pred_onehot = mlp_onehot.predict(X_test_onehot)
12 onehot_time = time.time() - start_time
13 print(f" Prediction Time (One-Hot Encoding): {onehot_time:.4f} seconds")
14
15 # Measure prediction time for TF-IDF
16 start_time = time.time()
17 y_test_pred_tfidf = mlp_tfidf.predict(X_test_tfidf)
18 tfidf_time = time.time() - start_time
19 print(f" Prediction Time (TF-IDF): {tfidf_time:.4f} seconds")

```

Prediction Time (BoW): 0.0379 seconds

Prediction Time (One-Hot Encoding): 0.0181 seconds

Prediction Time (TF-IDF): 0.0222 seconds

6.4 Conclusion

Les résultats obtenus montrent que la méthode **TF-IDF** est la plus performante pour la classification d'auteurs :

- **TF-IDF** atteint une **précision de 78.07%**, surpassant **BoW (75.80%)** et **One-Hot Encoding (75.63%)**.
- Le **score F1** et la **moyenne macro** de TF-IDF sont également les meilleurs, garantissant un bon équilibre entre les classes.
- En termes de rapidité d'exécution, TF-IDF est légèrement plus lent que One-Hot Encoding (**0.0222s contre 0.0181s**), mais reste très efficace.

Conclusion : La représentation **TF-IDF** est le meilleur choix dans ce contexte, car elle met en évidence les mots les plus discriminants pour chaque auteur, tout en atténuant l'impact des mots courants.

7 G. Vectorisations basées sur les embeddings de mots

7.1 1. Word2Vec

```
1 import gensim
2 from gensim.models import Word2Vec
3 import nltk
4 from nltk.tokenize import word_tokenize
5 import pandas as pd
6
7 sentences = df['cleaned_text'].dropna().apply(word_tokenize).tolist()
8
9 w2v_cbow = Word2Vec(sentences, vector_size=100, window=5, min_count=2, workers=4, sg=0)
10 w2v_skipgram = Word2Vec(sentences, vector_size=100, window=5, min_count=2, workers=4, sg=1)
11
12 w2v_cbow.save("word2vec_cbow.model")
13 w2v_skipgram.save("word2vec_skipgram.model")
14
15 print(w2v_cbow.wv['ghost'])
16 print(w2v_cbow.wv.most_similar('beauty', topn=5))
17
18 import numpy as np
19
20 def get_sentence_vector(sentence, model):
21     words = word_tokenize(sentence)
22     vectors = [model.wv[word] for word in words if word in model.wv]
23     return np.mean(vectors, axis=0) if vectors else np.zeros(100)
24
25 X_train_vectors_cbow = np.array([get_sentence_vector(text, w2v_cbow) for text in X_train['
    cleaned_text']])
26 X_test_vectors_cbow = np.array([get_sentence_vector(text, w2v_cbow) for text in X_test['
    cleaned_text']])
27
28 X_train_vectors_skip = np.array([get_sentence_vector(text, w2v_skipgram) for text in X_train['
    cleaned_text']])
29 X_test_vectors_skip = np.array([get_sentence_vector(text, w2v_skipgram) for text in X_test['
    cleaned_text']])
```

7.2 2. GloVe

```
1 import numpy as np
2
3 glove_path = "glove.6B.100d.txt"
4 glove_embeddings = {}
5
6 with open(glove_path, "r", encoding="utf-8") as f:
7     for line in f:
8         values = line.split()
9         word = values[0]
10        vector = np.array(values[1:], dtype=np.float32)
11        glove_embeddings[word] = vector
12
13 from sklearn.metrics.pairwise import cosine_similarity
14
15 def find_similar_words(word, embeddings, top_n=5):
16     if word not in embeddings:
17         return f"'{word}' not found in GloVe."
18
19     word_vector = embeddings[word].reshape(1, -1)
20     similarities = {}
21
22     for other_word, other_vector in embeddings.items():
23         if other_word != word:
24             sim = cosine_similarity(word_vector, other_vector.reshape(1, -1))[0][0]
25             similarities[other_word] = sim
```

```

26     similar_words = sorted(similarities.items(), key=lambda x: x[1], reverse=True)[:top_n]
27     return similar_words
28
29
30 print(find_similar_words('haunted', glove_embeddings))
31
32 def get_sentence_vector(sentence, embeddings, dim=100):
33     words = word_tokenize(sentence)
34     vectors = [embeddings[word] for word in words if word in embeddings]
35     return np.mean(vectors, axis=0) if vectors else np.zeros(dim)
36
37 X_train_vectors_glove = np.array([get_sentence_vector(text, glove_embeddings) for text in X_train
38     ['cleaned_text']])
39 X_test_vectors_glove = np.array([get_sentence_vector(text, glove_embeddings) for text in X_test['
40     cleaned_text']])

```

7.3 3. FastText

```

1 from gensim.models import FastText
2 from nltk.tokenize import word_tokenize
3
4 fasttext_model = FastText(sentences, vector_size=100, window=5, min_count=2, workers=4)
5 fasttext_model.save("fasttext.model")
6
7 import numpy as np
8
9 def get_fasttext_sentence_vector(sentence, model):
10     words = word_tokenize(sentence)
11     vectors = [model.wv[word] for word in words if word in model.wv]
12     return np.mean(vectors, axis=0) if vectors else np.zeros(100)
13
14 X_train_vectors_fasttext = np.array([get_fasttext_sentence_vector(text, fasttext_model) for text
15     in X_train['cleaned_text']])
16 X_test_vectors_fasttext = np.array([get_fasttext_sentence_vector(text, fasttext_model) for text
17     in X_test['cleaned_text']])

```

8 H. Entrainement / Test

8.1 1. Les mêmes questions des sections E et F mais avec les nouvelles représentations vectorielles présentées dans la section G

```
1 from sklearn.neural_network import MLPClassifier
2 from sklearn.metrics import accuracy_score, classification_report
3 from sklearn.preprocessing import StandardScaler
4 import numpy as np
5
6 mlp_w2v_cbow = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=0)
7 mlp_w2v_skip = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=0)
8 mlp_glove = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=0)
9 mlp_fasttext = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=0)
10
11 scaler = StandardScaler()
12 X_train_vectors_cbow = scaler.fit_transform(X_train_vectors_cbow)
13 X_test_vectors_cbow = scaler.transform(X_test_vectors_cbow)
14
15 X_train_vectors_skip = scaler.fit_transform(X_train_vectors_skip)
16 X_test_vectors_skip = scaler.transform(X_test_vectors_skip)
17
18 X_train_vectors_glove = scaler.fit_transform(X_train_vectors_glove)
19 X_test_vectors_glove = scaler.transform(X_test_vectors_glove)
20
21 X_train_vectors_fasttext = scaler.fit_transform(X_train_vectors_fasttext)
22 X_test_vectors_fasttext = scaler.transform(X_test_vectors_fasttext)
23
24 # Train models
25 mlp_w2v_cbow.fit(X_train_vectors_cbow, y_train)
26 mlp_w2v_skip.fit(X_train_vectors_skip, y_train)
27 mlp_glove.fit(X_train_vectors_glove, y_train)
28 mlp_fasttext.fit(X_train_vectors_fasttext, y_train)
29
30 def predict_on_train(model, X_train, y_train, name):
31     y_train_pred = model.predict(X_train)
32     accuracy = model.score(X_train, y_train)
33
34     print(f"\nMLP avec {name} - Accuracy sur l'entra nement: {accuracy:.4f}")
35     print(classification_report(y_train, y_train_pred))
36
37 predict_on_train(mlp_w2v_cbow, X_train_vectors_cbow, y_train, "Word2Vec CBOW")
38 predict_on_train(mlp_w2v_skip, X_train_vectors_skip, y_train, "Word2Vec Skip-gram")
39 predict_on_train(mlp_glove, X_train_vectors_glove, y_train, "GloVe")
40 predict_on_train(mlp_fasttext, X_train_vectors_fasttext, y_train, "FastText")
41
42 def predict_on_test(model, X_test, y_test, name):
43     y_test_pred = model.predict(X_test)
44     accuracy = model.score(X_test, y_test)
45
46     print(f"\nMLP avec {name} - Accuracy sur le test: {accuracy:.4f}")
47     print(classification_report(y_test, y_test_pred, zero_division=1)) # Avoid division errors
48
49 # Run predictions on test data
50 predict_on_test(mlp_w2v_cbow, X_test_vectors_cbow, y_test, "Word2Vec CBOW")
51 predict_on_test(mlp_w2v_skip, X_test_vectors_skip, y_test, "Word2Vec Skip-gram")
52 predict_on_test(mlp_glove, X_test_vectors_glove, y_test, "GloVe")
53 predict_on_test(mlp_fasttext, X_test_vectors_fasttext, y_test, "FastText")
```

9 Comparaison des modèles

Les modèles suivants ont été comparés en utilisant différentes méthodes de vectorisation :

- **BoW (Bag of Words)** : Accuracy de **0.7580**.
- **One-Hot Encoding** : Accuracy de **0.7563**.
- **TF-IDF** : Accuracy de **0.7807**.
- **Word2Vec CBOW** : Accuracy de **0.5424**.
- **Word2Vec Skip-gram** : Accuracy de **0.6483**.
- **GloVe** : Accuracy de **0.6228**.
- **FastText** : Accuracy de **0.5574**.

10 Analyse des méthodes de vectorisation

Méthodes traditionnelles

- **BoW** : Compte la fréquence des mots. Simple mais ignore la sémantique.
- **One-Hot Encoding** : Représentation binaire des mots. Moins efficace pour les grands corpus.
- **TF-IDF** : Pondere la fréquence des mots par leur importance. Meilleure performance (**0.35**).

Méthodes basées sur les embeddings

- **Word2Vec** : Capture les relations sémantiques. Performance inférieure (**0.5424**).
- **GloVe** : Combine factorisation matricielle et contexte local. Performance similaire à Word2Vec.
- **FastText** : Prend en compte les sous-mots. Performance légèrement inférieure.

11 Conclusion

La méthode **TF-IDF** est la plus adaptée pour cette tâche de classification d'auteurs. Elle combine simplicité et efficacité en pondérant les mots selon leur importance dans le corpus. Les méthodes basées sur les embeddings, bien que puissantes pour capturer des relations sémantiques, n'ont pas surpassé les méthodes traditionnelles dans ce contexte.

12 Justification

- **TF-IDF** est efficace pour capturer les motifs lexicaux spécifiques à chaque auteur.
- Les **embeddings** sont plus adaptés pour des tâches où la sémantique et les relations contextuelles sont primordiales.